

Image Recognition Model Using Google Teachable Machine

Prepared by: Raghad Alhamad

1. Project Objective

The objective of this project is to build an image recognition model using Google Teachable Machine. The model is trained to classify different categories of waste materials. After training, the model is exported in TensorFlow Keras format and tested using a Python script.

2. Dataset

The training dataset was obtained from the TrashNet Dataset available on Kaggle.

Dataset Name: TrashNet Dataset

Platform: Kaggle

Source: <https://www.kaggle.com/datasets/feyzazkefe/trashnet>

The dataset contains images of different waste categories:

- Cardboard
- Glass
- Metal
- Paper
- Plastic
- Trash

3. Creating the Image Recognition Model

Google Teachable Machine was used to create the image classification model. The dataset images were uploaded into separate classes, and the model was trained until the training process was completed successfully.

4. Exporting the Model

After training, the model was exported using TensorFlow Keras format. The exported files were:

- keras_model.h5
- labels.txt

These files allow the trained model to be loaded and used in Python.

5. Python Implementation

A Python script was created to load the exported model and predict the class of an input image. The script performs the following steps:

- Load the trained Keras model.
- Read class labels.
- Load and preprocess the input image.
- Resize the image to 224x224 pixels.
- Normalize image values.
- Predict the image category and confidence score.

6. Model Testing

The model was tested using a new image that was not included during training. The prediction result was:

Predicted Class: Cardboard

Confidence Score: 99.98%

7. GitHub Submission Files

The following files were uploaded to the GitHub repository:

- predict.py
- keras_model.h5
- labels.txt
- Test image
- Output screenshot
- README.md

8. Conclusion

This project demonstrates the complete workflow of developing an image recognition model using Google Teachable Machine. The process included dataset preparation, model training, exporting the TensorFlow Keras model, Python implementation, and model evaluation.

Project Screenshots

Figure 1: Project Requirements



Task Instructions:

1. Train an **image recognition model** using **Teachable Machine by Google** Use at least two classes and evaluate the model.
2. Download the trained model in **TensorFlow → Keras format**.
3. Write a Python script that **loads** the model, accepts an **input image**, and predicts its **class**.
4. Submit the **Python script**, the **exported model files**, and a **screenshot** of the output.

ارفاق GitHub الملفات مع شرح جميع الخطوات على

Figure 2: Training the model in Google Teachable Machine

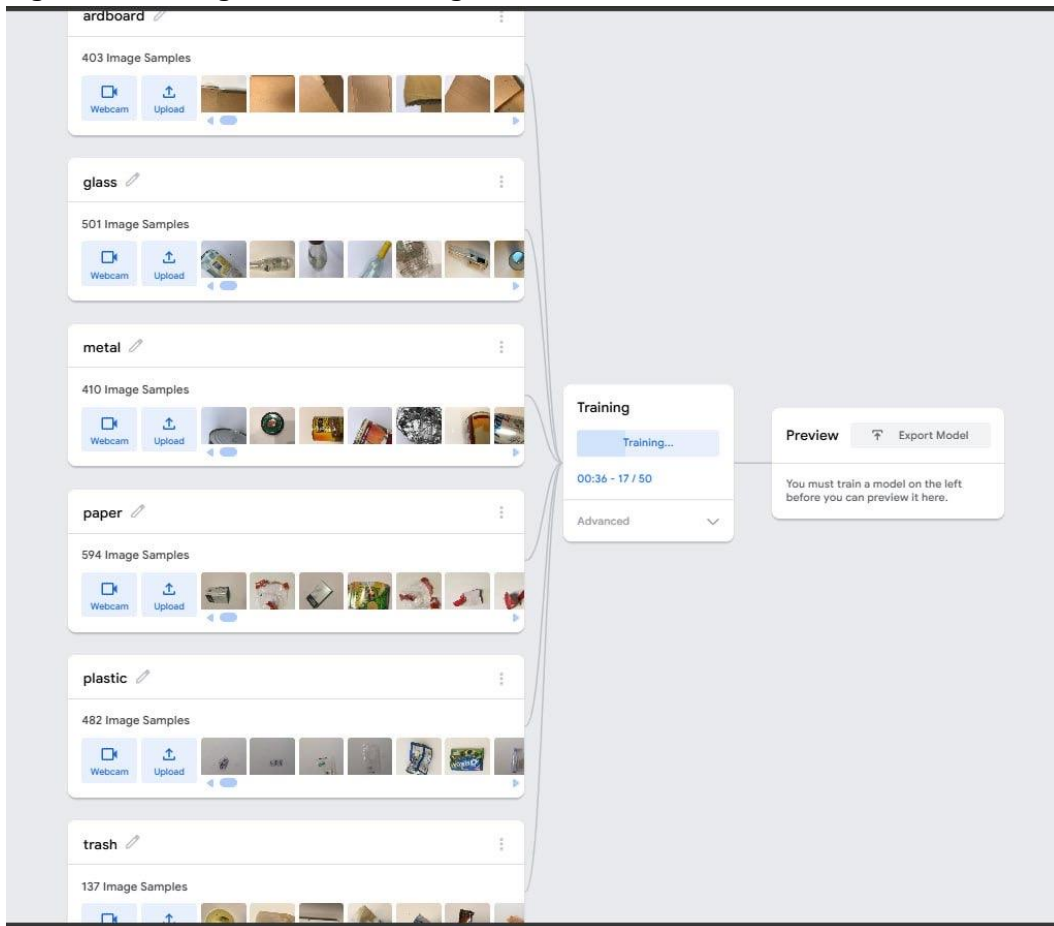


Figure 3: Exporting the model in TensorFlow Keras format

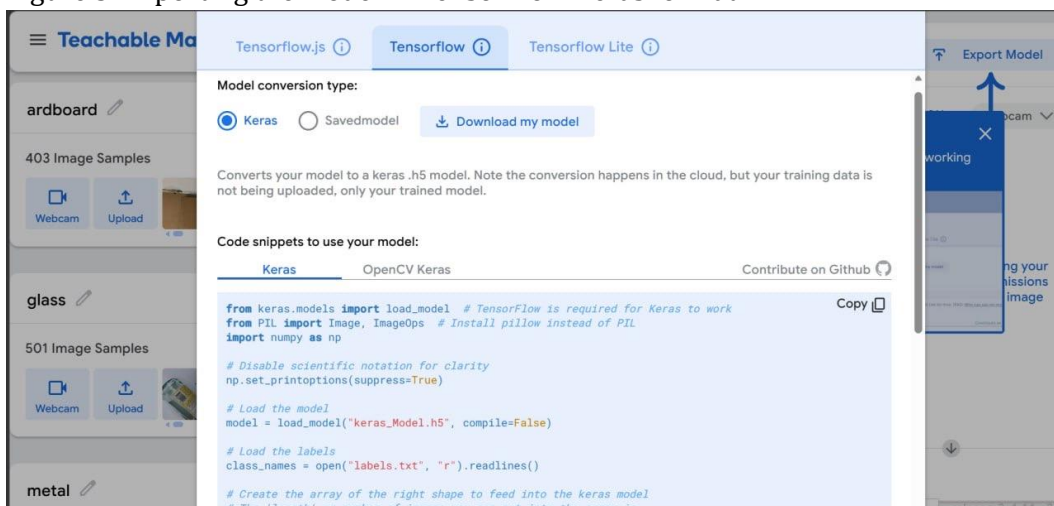
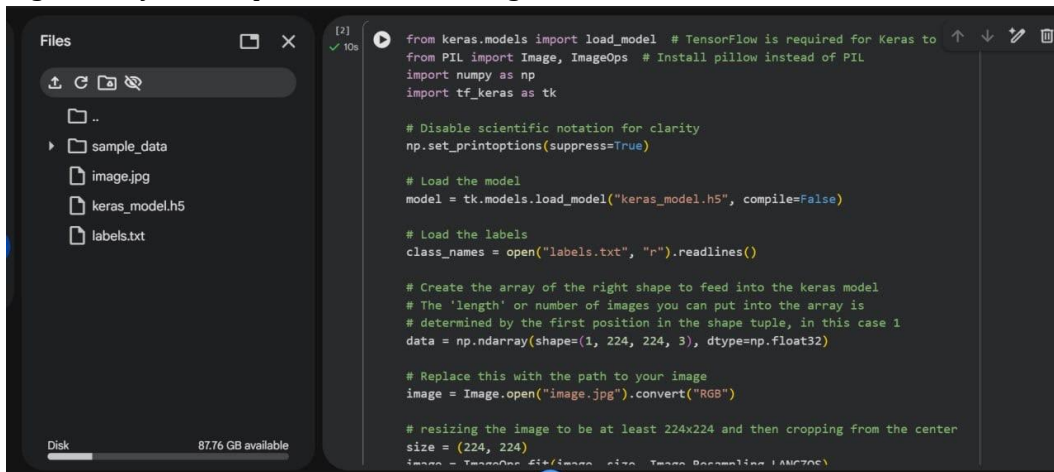


Figure 4: Python implementation in Google Colab



```
[?] 10s
from keras.models import load_model # TensorFlow is required for Keras to
from PIL import Image, ImageOps # Install pillow instead of PIL
import numpy as np
import tf_keras as tk

# Disable scientific notation for clarity
np.set_printoptions(suppress=True)

# Load the model
model = tk.models.load_model("keras_model.h5", compile=False)

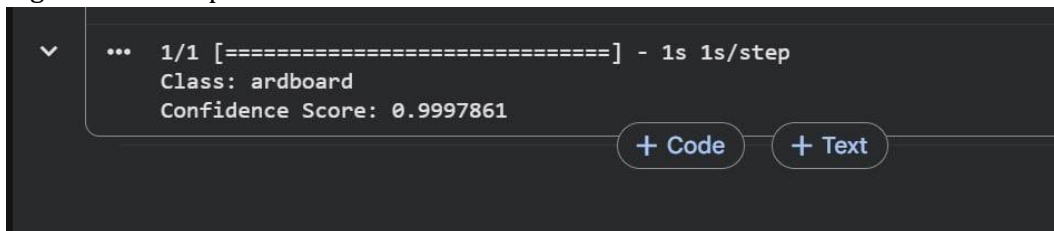
# Load the labels
class_names = open("labels.txt", "r").readlines()

# Create the array of the right shape to feed into the keras model
# The 'length' or number of images you can put into the array is
# determined by the first position in the shape tuple, in this case 1
data = np.ndarray(shape=(1, 224, 224, 3), dtype=np.float32)

# Replace this with the path to your image
image = Image.open("image.jpg").convert("RGB")

# Resizing the image to be at least 224x224 and then cropping from the center
size = (224, 224)
image = ImageOps.fit(image, size, Image.Resampling.LANCZOS)
```

Figure 5: Model prediction result



```
... 1/1 [=====] - 1s 1s/step
Class: ardbord
Confidence Score: 0.9997861
```

+ Code + Text